

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÀI GIẢNG
LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

NGUYỄN MẠNH SƠN

HÀ NỘI 2020

MỤC LỤC

MỤC LỤC	2
GIỚI THIỆU	7
PHẦN 1: NHỮNG KHÁI NIỆM CƠ BẢN	9
CHƯƠNG 1: TỔNG QUAN VỀ CÁCH TIẾP CẬN HƯỚNG ĐỐI TƯỢNG	10
1.1 PHƯƠNG PHÁP TIẾP CẬN CỦA LẬP TRÌNH TRUYỀN THỐNG	10
1.1.1 Lập trình tuyến tính	10
1.1.2 Lập trình cấu trúc	11
1.2 PHƯƠNG PHÁP TIẾP CẬN HƯỚNG ĐỐI TƯỢNG	13
1.2.1 Phương pháp lập trình hướng đối tượng	13
1.2.2 Phương pháp phân tích và thiết kế hướng đối tượng	15
1.3 SO SÁNH HAI CÁCH TIẾP CẬN	17
1.4 XU HƯỚNG PHÁT TRIỂN CỦA LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG	18
TỔNG KẾT CHƯƠNG 1	20
CHƯƠNG 2: NHỮNG KHÁI NIỆM CƠ BẢN CỦA LTHDT	21
2.1 CÁC KHÁI NIỆM CƠ BẢN	21
2.1.1 Đối tượng	21
2.1.2 Lớp đối tượng	22
2.1.3 Trừu tượng hoá đối tượng theo chức năng	24
2.1.4 Trừu tượng hoá đối tượng theo dữ liệu	25
2.1.5 Khái niệm kế thừa	26
2.1.6 Khái niệm đóng gói	28
2.1.7 Khái niệm đa hình	29
2.2 SO SÁNH LỚP VÀ CẤU TRÚC	30
2.3 THÀNH PHẦN PRIVATE VÀ PUBLIC CỦA LỚP	31
2.4 MỘT SỐ NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG	32
2.4.1 C++	32
2.4.2 ASP.NET và C#.NET	33
2.4.3 Java	33
TỔNG KẾT CHƯƠNG 2	34
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 2	35
PHẦN 2: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI JAVA	36

CHƯƠNG 3: GIỚI THIỆU VỀ JAVA.....	37
3.1 LỊCH SỬ PHÁT TRIỂN CỦA JAVA.....	37
3.1.1 Java.....	37
3.1.2 Đặc trưng của ngôn ngữ Java	37
3.1.3 Cài đặt Java.....	40
3.2 KIẾN TRÚC CHƯƠNG TRÌNH XÂY DỰNG TRÊN JAVA	41
3.2.1 Kiến trúc chương trình Java.....	41
3.2.2 Chương trình Java đầu tiên.....	44
3.2.3 Phân tích chương trình đầu tiên.....	45
3.3 CÁC KIỂU DỮ LIỆU VÀ TOÁN TỬ CƠ BẢN TRÊN JAVA	47
3.3.1 Khai báo biến.....	47
3.3.2 Kiểu dữ liệu	47
3.3.3 Các toán tử.....	49
3.4 CÁC CẤU TRÚC LỆNH TRÊN JAVA.....	53
3.4.1 Câu lệnh if-else	53
3.4.2 Câu lệnh switch-case	54
3.4.3 Vòng lặp While.....	56
3.4.4 Vòng lặp do-while	57
3.4.5 Vòng lặp for.....	58
3.5 CASE STUDY I	59
TỔNG KẾT CHƯƠNG 3	61
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 3	62
CHƯƠNG 4: KẾ THỪA VÀ ĐA HÌNH TRÊN JAVA	65
4.1 KẾ THỪA ĐƠN	65
4.1.1 Lớp.....	65
4.1.2 Sự kế thừa	70
4.2 KẾ THỪA BỘI.....	73
4.2.1 Giao tiếp	73
4.2.2 Sử dụng giao tiếp	74
4.3 LỚP TRỪU TƯỢNG	76
4.3.1 Khai báo.....	76
4.3.2 Sử dụng lớp trừu tượng.....	78
4.4 ĐA HÌNH	79
4.4.1 Nạp chồng.....	80
4.4.2 Đa hình.....	80

4.5 CASE STUDY II.....	82
4.5.1 Lớp Human.....	82
4.5.2 Lớp Person.....	83
4.5.3 Lớp Employee	84
4.5.4 Chương trình demo.....	86
TỔNG KẾT CHƯƠNG 4.....	87
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 4	88
CHƯƠNG 5: BIỂU DIỄN VÀ CÀI ĐẶT CÁC CTDL TRÊN JAVA.....	93
5.1 PHƯƠNG PHÁP DUYỆT VÀ ĐỆ QUI	93
5.1.1 Các phương pháp duyệt.....	93
5.1.2 Phương pháp đệ qui	94
5.2 PHƯƠNG PHÁP SẮP XẾP VÀ TÌM KIẾM.....	94
5.2.1 Các phương pháp sắp xếp.....	94
5.2.2 Các phương pháp tìm kiếm.....	96
5.3 NGĂN XẾP VÀ HÀNG ĐỢI.....	98
5.3.1 Ngăn xếp.....	98
5.3.2 Hàng đợi	100
5.4 DANH SÁCH LIÊN KẾT	102
5.4.1 Danh sách liên kết đơn	102
5.4.2 Danh sách liên kết kép.....	108
5.5 CÂY NHỊ PHÂN.....	113
5.6 ĐỒ THỊ.....	119
5.6.1 Biểu diễn đồ thị.....	119
5.6.2 Cài đặt đồ thị không có trọng số.....	121
5.6.3 Cài đặt đồ thị có trọng số.....	126
5.7 CASE STUDY III.....	131
TỔNG KẾT CHƯƠNG 5	137
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 5	137
CHƯƠNG 6: LẬP TRÌNH GIAO DIỆN TRÊN JAVA.....	139
6.1 GIAO DIỆN VỚI CÁC ĐỐI TƯỢNG CƠ BẢN	139
6.1.1 Các đối tượng container cơ bản	139
6.1.2 Các đối tượng component cơ bản	142
6.1.3 Các sự kiện cơ bản của đối tượng.....	146
6.2 GIAO DIỆN VỚI CÁC ĐỐI TƯỢNG MULTIMEDIA	150
6.2.1 Ô đánh dấu và nút chọn	150

6.2.2 Lựa chọn	152
6.2.3 Danh sách	154
6.2.4 Trình đơn	157
6.3 CÁC KỸ THUẬT TẠO TABLES	160
6.3.1 Trình bày Flow Layout	161
6.3.2 Trình bày Grid Layout	162
6.3.3 Trình bày Border Layout	163
6.3.4 Trình bày GridBag Layout	164
6.3.5 Trình bày Null Layout	167
6.4 HTML & APPLET	168
6.4.1 Cấu trúc của một Applet	168
6.4.2 Sử dụng applet	170
6.4.3 Truyền tham số cho Applet	173
6.5 GIỚI THIỆU VỀ SWING	174
6.5.1 Mở rộng các đối tượng component	175
6.5.2 Mở rộng các đối tượng container	176
6.6 CASE STUDY IV	179
TỔNG KẾT CHƯƠNG 6	186
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 6	187
CHƯƠNG 7: LẬP TRÌNH GENERIC.....	188
7.1 Giới thiệu về Generic:	189
7.1.1 Đặt vấn đề	189
7.1.2. Khái niệm Generic	189
7.2 Các phương thức Generic trong Java:	190
7.3 Các lớp Generic Classes	192
7.4 Các kí tự đại diện Generics	195
7.5 Ưu và một số lưu ý khi sử dụng Gerenics.	197
7.5.1 Ưu điểm	197
7.5.2 Một số lưu ý khi sử dụng Gerenics:	197
CHƯƠNG 8: THƯ VIỆN CÁC COLLECTION TRONG JAVA VÀ ÁP DỤNG.	199
8.1 Các thành phần của Collection	200
8.2 Giới thiệu Collection:	201
8.3 Duyệt Collection:	202
8.3.1 Sử dụng for – each:	202
8.3.2 Sử dụng Iterator:	202

8.2.3 Thao tác số lượng lớn (Bulk)	203
8.2.4 Thao tác mảng (array):	205
8.3 Giới thiệu List và cách sử dụng	206
8.3.1 So sánh với Vector	207
8.3.2 ArrayList	208
8.3.3 Danh sách liên kết (Linked List)	208
8.3.4 Giới thiệu Queue:	211
8.4 Giới thiệu Set Interface:	216
8.5 Giới thiệu về Map	223
TÀI LIỆU THAM KHẢO	230
PHỤ LỤC: BÀI TẬP LẬP TRÌNH JAVA	231

GIỚI THIỆU

Trong những năm gần đây, lập trình hướng đối tượng đã trở nên gần gũi nhờ sự ra đời liên tiếp của các ngôn ngữ lập trình hướng đối tượng. Sức mạnh của phương pháp lập trình hướng đối tượng thể hiện ở chỗ khả năng mô hình hoá hệ thống dựa trên các đối tượng thực tế, khả năng đóng gói và bảo vệ an toàn dữ liệu, khả năng sử dụng lại mã nguồn để tiết kiệm chi phí và tài nguyên; đặc biệt là khả năng chia sẻ mã nguồn trong cộng đồng lập trình viên chuyên nghiệp. Những điểm mạnh này hứa hẹn sẽ thúc đẩy phát triển một môi trường lập trình tiên tiến cùng với nền công nghiệp lắp ráp phần mềm với các thư viện thành phần có sẵn.

Tài liệu này nhằm giới thiệu cho các sinh viên một cái nhìn tổng quan về phương pháp lập trình hướng đối tượng cùng cung cấp những kiến thức, các kỹ thuật cơ bản cho phát triển các ứng dụng của mình dựa trên ngôn ngữ lập trình Java - một trong những ngôn ngữ lập trình hướng đối tượng thông dụng nhất hiện nay.

Nội dung của tài liệu này bao gồm hai phần chính:

Phần thứ nhất trình bày những khái niệm và các vấn đề cơ bản của lập trình hướng đối tượng bao gồm tổng quan về cách tiếp cận hướng đối tượng và các khái niệm đối tượng, lớp, kế thừa, đóng gói, đa hình...

Phần thứ hai trình bày chi tiết phương pháp lập trình hướng đối tượng với ngôn ngữ lập trình Java.

Nội dung của tài liệu bao gồm 7 chương:

Chương 1: Tổng quan về cách tiếp cận hướng đối tượng. Trình bày sự tiến hoá của cách tiếp cận từ lập trình truyền thống đến cách tiếp cận của lập trình hướng đối tượng và xu hướng phát triển của lập trình hướng đối tượng hiện nay.

Chương 2: Những khái niệm cơ bản của lập trình hướng đối tượng. Trình bày các khái niệm cơ bản như: đối tượng, lớp đối tượng với các thuộc tính và phương thức, tính kế thừa và đa hình, tính đóng gói của lập trình hướng đối tượng. Chương này cũng giới thiệu tổng quan một số ngôn ngữ lập trình hướng đối tượng thông dụng hiện nay.

Chương 3: Ngôn ngữ Java. Giới thiệu những khái niệm và những quy ước ban đầu của ngôn ngữ lập trình Java: Cấu trúc chương trình, cách biên dịch, cách đặt tên biến, kiểu dữ liệu, các toán tử và cấu trúc lệnh của ngôn ngữ Java.

Chương 4: Kế thừa và đa hình trên Java. Trình bày các kỹ thuật lập trình hướng đối tượng dựa trên ngôn ngữ Java: Khai báo lớp, các thuộc tính và phương thức của lớp; kỹ thuật thừa kế, các lớp trừu tượng, cài đặt nạp chồng và đa hình trên Java.

Chương 5: Biểu diễn và cài đặt các cấu trúc dữ liệu trừu tượng trên Java. Trình bày kỹ thuật cài đặt và sử dụng một số cấu trúc dữ liệu quen thuộc trong Java: ngăn xếp, hàng đợi, danh sách liên kết, cây nhị phân và đồ thị.

Chương 6: Lập trình giao diện trên Java. Trình bày các kỹ thuật lập trình giao diện trên Java: Lập trình với các giao diện cơ bản trong thư viện AWT, lập trình giao diện với Applet và HTML, lập trình giao diện nâng cao với thư viện SWING.

Chương 7: Lập trình Generic. Trình bày về lập trình khái quát (Generic), cách dùng và các trường hợp áp dụng.

Chương 8: Thư viện các Collection và áp dụng: Giới thiệu các lớp trong gói thư viện Collection của Java, cấu trúc, các phương thức và cách sử dụng.

Tài liệu này được viết nhằm phục vụ môn học “**Lập trình hướng đối tượng**” giảng dạy tiếp theo sau môn học Ngôn ngữ lập trình C++ và như vậy khi học môn học này sinh viên sẽ dễ nắm bắt được những đặc trưng khác biệt của ngôn ngữ Java so với C++.

Mặc dù các tác giả đã có nhiều cố gắng trong quá trình biên soạn tài liệu này, song không thể tránh khỏi những thiếu sót. Rất mong nhận được sự đóng góp ý kiến của sinh viên và các bạn đồng nghiệp.

PHẦN 1

NHỮNG KHÁI NIỆM CƠ BẢN
CỦA LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

CHƯƠNG 1

TỔNG QUAN VỀ CÁCH TIẾP CẬN HƯỚNG ĐỐI TƯỢNG

Nội dung chương này nhằm giới thiệu một cách tổng quan về cách tiếp cận hướng đối tượng. Nội dung trình bày bao gồm:

Giới thiệu về cách tiếp cận của lập trình truyền thống.

Giới thiệu cách tiếp cận của lập trình hướng đối tượng.

So sánh sự khác biệt giữa hai cách tiếp cận này.

Xu hướng hiện nay của lập trình hướng đối tượng

1.1 PHƯƠNG PHÁP TIẾP CẬN CỦA LẬP TRÌNH TRUYỀN THỐNG

Lập trình truyền thống đã trải qua hai giai đoạn:

Giai đoạn sơ khai, khi khái niệm lập trình mới ra đời, là lập trình tuyến tính.

Giai đoạn tiếp theo, là lập trình hướng cấu trúc.

1.1.1 Lập trình tuyến tính

Đặc trưng cơ bản của lập trình tuyến tính là tư duy theo lối tuần tự. Chương trình sẽ được thực hiện tuần tự từ đầu đến cuối, lệnh này kế tiếp lệnh kia cho đến khi kết thúc chương trình.

Đặc trưng

Lập trình tuyến tính có hai đặc trưng:

Đơn giản: chương trình được tiến hành đơn giản theo lối tuần tự, không phức tạp.

Đơn luồng: chỉ có một luồng công việc duy nhất, và các công việc được thực hiện tuần tự trong luồng đó.

Tính chất

Ưu điểm: Do tính đơn giản, lập trình tuyến tính có ưu điểm là chương trình đơn giản, dễ hiểu. Lập trình tuyến tính được ứng dụng cho các chương trình đơn giản.

Nhược điểm: Với các ứng dụng phức tạp, người ta không thể dùng lập trình tuyến tính để giải quyết.

Ngày nay, lập trình tuyến tính chỉ tồn tại trong phạm vi các modul nhỏ nhất của các phương pháp lập trình khác. Ví dụ trong một chương trình con của lập trình cấu trúc, các lệnh cũng được thực hiện theo tuần tự từ đầu đến cuối chương trình con.

1.1.2 Lập trình cấu trúc

Trong lập trình hướng cấu trúc, chương trình chính được chia nhỏ thành các chương trình con và mỗi chương trình con thực hiện một công việc xác định. Chương trình chính sẽ gọi đến chương trình con theo một giải thuật, hoặc một cấu trúc được xác định trong chương trình chính.

Các ngôn ngữ lập trình cấu trúc phổ biến là Pascal, C và C++. Riêng C++ ngoài việc có đặc trưng của lập trình cấu trúc do kế thừa từ C, còn có đặc trưng của lập trình hướng đối tượng. Cho nên C++ còn được gọi là ngôn ngữ lập trình nửa cấu trúc, nửa hướng đối tượng.

Đặc trưng

Đặc trưng cơ bản nhất của lập trình cấu trúc thể hiện ở mối quan hệ:

Chương trình = Cấu trúc dữ liệu + Giải thuật

Trong đó:

Cấu trúc dữ liệu là cách tổ chức dữ liệu, cách mô tả bài toán dưới dạng ngôn ngữ lập trình

Giải thuật là một quy trình để thực hiện một công việc xác định

Trong chương trình, giải thuật có quan hệ phụ thuộc vào cấu trúc dữ liệu:

Một cấu trúc dữ liệu chỉ phù hợp với một số hạn chế các giải thuật.

Nếu thay đổi cấu trúc dữ liệu thì phải thay đổi giải thuật cho phù hợp.

Một giải thuật thường phải đi kèm với một cấu trúc dữ liệu nhất định.

Tính chất

Mỗi chương trình con có thể được gọi thực hiện nhiều lần trong một chương trình chính.

Các chương trình con có thể được gọi đến để thực hiện theo một thứ tự bất kì, tùy thuộc vào giải thuật trong chương trình chính mà không phụ thuộc vào thứ tự khai báo của các chương trình con.

Các ngôn ngữ lập trình cấu trúc cung cấp một số cấu trúc lệnh điều khiển chương trình.

Ưu điểm

Chương trình sáng sủa, dễ hiểu, dễ theo dõi.

Tư duy giải thuật rõ ràng.

Nhược điểm

Lập trình cấu trúc không hỗ trợ việc sử dụng lại mã nguồn: Giải thuật luôn phụ thuộc chặt chẽ vào cấu trúc dữ liệu, do đó, khi thay đổi cấu trúc dữ liệu, phải thay đổi giải thuật, nghĩa là phải viết lại chương trình.

Không phù hợp với các phần mềm lớn: tư duy cấu trúc với các giải thuật chỉ phù hợp với các bài toán nhỏ, nằm trong phạm vi một modul của chương trình. Với dự án phần mềm lớn, lập trình cấu trúc tỏ ra không hiệu quả trong việc giải quyết mối quan hệ vĩ mô giữa các modul của phần mềm.

Vấn đề

Vấn đề cơ bản của lập trình cấu trúc là bằng cách nào để phân chia chương trình chính thành các chương trình con cho phù hợp với yêu cầu, chức năng và mục đích của mỗi bài toán.

Thông thường, để phân rã bài toán trong lập trình cấu trúc, người ta sử dụng phương pháp thiết kế trên xuống (top-down).

Phương pháp thiết kế trên xuống (top-down)

Phương pháp thiết kế top-down tiếp cận bài toán theo hướng từ trên xuống dưới, từ tổng quan đến chi tiết. Theo đó, một bài toán được chia thành các bài toán con nhỏ hơn. Mỗi bài toán con lại được chia nhỏ tiếp, nếu có thể, thành các bài toán con nhỏ hơn nữa.

Quá trình này còn được gọi là quá trình làm mịn dần. Quá trình làm mịn dần sẽ dừng lại khi các bài toán con không cần chia nhỏ thêm nữa. Nghĩa là khi mỗi bài toán con đều có thể giải quyết bằng một chương trình con với một giải thuật đơn giản.

Ví dụ, sử dụng phương pháp top-down để giải quyết bài toán là xây một căn nhà mới. Khi đó, ta có thể phân rã bài toán theo các bước như sau:

Ở mức thứ nhất, chia bài toán xây nhà thành các bài toán nhỏ hơn như: làm móng, đổ cột, đổ trần, xây tường, lợp mái.

Ở mức thứ hai, phân rã các công việc ở mức thứ nhất: việc làm móng nhà có thể phân rã tiếp thành các công việc: đào móng, gia cố nền, làm khung sắt, đổ bê tông. Công việc đổ cột được phân rã thành ...

Ở mức thứ ba, phân rã các công việc của mức thứ hai: việc đào móng có thể phân chia tiếp thành các công việc: đo đạc, cắm mốc, căng dây, đào và kiểm tra móng. Việc gia cố nền được phân rã thành ...

Quá trình phân rã có thể dừng ở mức này, bởi vì các công việc con thu được là: đo đạc, cắm mốc, căng dây, đào... có thể thực hiện được ngay, không cần chia nhỏ thêm nữa.

Lưu ý:

Cùng sử dụng phương pháp top-down với cùng một bài toán, nhưng có thể cho ra nhiều kết quả khác nhau. Nguyên nhân là do sự khác nhau trong tiêu chí để phân rã một bài toán thành các bài toán con.

Ví dụ, vẫn áp dụng phương pháp top-down để giải quyết bài toán xây nhà, nhưng nếu sử dụng một cách khác để phân chia bài toán, ta có thể thu được kết quả khác biệt so với phương pháp ban đầu:

Ở mức thứ nhất, chia bài toán xây nhà thành các bài toán nhỏ hơn như: làm phần gỗ, làm phần sắt, làm phần bê tông và làm phần gạch.

Ở mức thứ hai, phân rã các công việc ở mức thứ nhất: việc làm gỗ có thể chia thành các công việc như: xẻ gỗ, gia công gỗ, tạo khung, lắp vào nhà. Việc làm sắt có thể chia nhỏ thành...

Rõ ràng, với cách làm mịn thế này, ta sẽ thu được một kết quả khác hẳn với cách thức đã thực hiện ở phần trên.

1.2 PHƯƠNG PHÁP TIẾP CẬN HƯỚNG ĐỐI TƯỢNG

1.2.1 Phương pháp lập trình hướng đối tượng

Xuất phát từ hai hạn chế chính của phương pháp lập trình cấu trúc:

Không quản lý được sự thay đổi dữ liệu khi có nhiều chương trình cùng thay đổi một biến chung. Vấn đề này đặc biệt nghiêm trọng khi các ứng dụng ngày càng lớn, người ta không thể kiểm soát được sự truy nhập đến các biến dữ liệu chung.

Không tiết kiệm được tài nguyên con người: Giải thuật gắn liền với cấu trúc dữ liệu, nếu thay đổi cấu trúc dữ liệu, sẽ phải thay đổi giải thuật, và do đó, phải viết lại mã chương trình từ đầu.

Để khắc phục được hai hạn chế này khi giải quyết các bài toán lớn, người ta xây dựng một phương pháp tiếp cận mới, là phương pháp lập trình hướng đối tượng, với hai mục đích chính:

Đóng gói dữ liệu để hạn chế sự truy nhập tự do vào dữ liệu, không quản lý được.

Cho phép sử dụng lại mã nguồn, hạn chế việc phải viết lại mã từ đầu cho các chương trình.

Việc đóng gói dữ liệu được thực hiện theo phương pháp trừu tượng hoá đối tượng thành lớp từ thấp lên cao như sau:

Thu thập các thuộc tính của mỗi đối tượng, gắn các thuộc tính vào đối tượng tương ứng.

Nhóm các đối tượng có các thuộc tính tương tự nhau thành nhóm, loại bỏ bớt các thuộc tính cá biệt, chỉ giữ lại các thuộc tính chung nhất. Đây được gọi là quá trình trừu tượng hoá đối tượng thành lớp.

Đóng gói dữ liệu của các đối tượng vào lớp tương ứng. Mỗi thuộc tính của đối tượng trở thành một thuộc tính của lớp tương ứng.

Việc truy nhập dữ liệu được thực hiện thông qua các phương thức được trang bị cho lớp. Không được truy nhập tự do trực tiếp đến dữ liệu.

Khi có thay đổi trong dữ liệu của đối tượng, ta chỉ cần thay đổi các phương thức truy nhập thuộc tính của lớp, mà không cần phải thay đổi mã nguồn của các chương trình sử dụng lớp tương ứng.

Việc cho phép sử dụng lại mã nguồn được thực hiện thông qua cơ chế kế thừa trong lập trình hướng đối tượng. Theo đó:

Các lớp có thể được kế thừa nhau để tận dụng các thuộc tính, các phương thức của nhau.

Trong lớp dẫn xuất (lớp được kế thừa) có thể sử dụng lại các phương thức của lớp cơ sở (lớp bị lớp khác kế thừa) mà không cần thiết phải cài đặt lại mã nguồn.

Ngay cả khi lớp dẫn xuất định nghĩa lại các phương thức cho mình, lớp cơ sở cũng không bị ảnh hưởng và không phải sửa lại bất kì một đoạn mã nguồn nào.

Ngôn ngữ lập trình hướng đối tượng phổ biến hiện nay là Java và C++. Tuy nhiên, C++ mặc dù cũng có những đặc trưng cơ bản của lập trình hướng đối tượng nhưng vẫn không phải là ngôn ngữ lập trình thuần hướng đối tượng. Java thật sự là một ngôn ngữ lập trình thuần hướng đối tượng.

Đặc trưng

Lập trình hướng đối tượng có hai đặc trưng cơ bản:

Đóng gói dữ liệu: dữ liệu luôn được tổ chức thành các thuộc tính của lớp đối tượng.

Việc truy nhập đến dữ liệu phải thông qua các phương thức của đối tượng lớp.

Sử dụng lại mã nguồn: việc sử dụng lại mã nguồn được thể hiện thông qua cơ chế kế thừa. Cơ chế này cho phép các lớp đối tượng có thể kế thừa từ các lớp đối tượng khác. Khi đó, trong các lớp kế thừa, có thể sử dụng các phương thức (mã nguồn) của các lớp bị kế thừa, mà không cần phải định nghĩa lại.

Ưu điểm

Lập trình hướng đối tượng có một số ưu điểm nổi bật:

Không còn nguy cơ dữ liệu bị thay đổi tự do trong chương trình. Vì dữ liệu đã được đóng gói vào các đối tượng. Nếu muốn truy nhập vào dữ liệu phải thông qua các phương thức cho phép của đối tượng.

Khi thay đổi cấu trúc dữ liệu của một đối tượng, không cần thay đổi các đối mã nguồn của các đối tượng khác, mà chỉ cần thay đổi một số hàm thành phần của đối tượng bị thay đổi. Điều này hạn chế sự ảnh hưởng xấu của việc thay đổi dữ liệu đến các đối tượng khác trong chương trình.

Có thể sử dụng lại mã nguồn, tiết kiệm tài nguyên. Vì nguyên tắc kế thừa cho phép các lớp kế thừa sử dụng các phương thức được kế thừa từ lớp khác như những phương thức của chính nó, mà không cần thiết phải định nghĩa lại.

Phù hợp với các dự án phần mềm lớn, phức tạp.

1.2.2 Phương pháp phân tích và thiết kế hướng đối tượng

Một vấn đề cơ bản đặt ra cho phương pháp hướng đối tượng là từ một bài toán ban đầu, làm sao để thu được một tập các đối tượng, với các chức năng được phối hợp với nhau, đáp ứng được yêu cầu của bài toán đặt ra?

Phương pháp phân tích thiết kế hướng đối tượng ra đời nhằm trả lời cho câu hỏi này. Mục đích là xây dựng một tập các lớp đối tượng tương ứng với mỗi bài toán, phương pháp này tiến hành theo hai pha chính:

Pha phân tích: Chuyển đổi yêu cầu bài toán từ ngôn ngữ tự nhiên sang ngôn ngữ mô hình.

Pha thiết kế: Chuyển đổi đặc tả bài toán dưới dạng ngôn ngữ mô hình sang một mô hình cụ thể có thể cài đặt được.

Hai pha phân tích và thiết kế này bao gồm nhiều bước khác nhau:

Mô tả bài toán

Đặc tả yêu cầu

Trích chọn đối tượng

Mô hình hoá lớp đối tượng

Thiết kế tổng quan

Thiết kế chi tiết.

Bước 1: Mô tả bài toán

Bài toán ban đầu được phát biểu dưới dạng ngôn ngữ tự nhiên, bao gồm:

Mục đích, chức năng chung

Các yêu cầu về thông tin dữ liệu

Các yêu cầu về chức năng thực hiện

Bước 2: Đặc tả yêu cầu

Các yêu cầu được hình thức hoá lên một mức cao hơn bằng cách sử dụng ngôn ngữ kịch bản (scenario) để mô tả. Trong một kịch bản, mỗi chức năng, mỗi hoạt động được mô tả bằng một kịch bản, bao gồm:

Các tác nhân tham gia vào kịch bản.

Vai trò của mỗi tác nhân trong kịch bản.

Thứ tự các hành động mà mỗi tác nhân thực hiện: khi nào thực hiện, tác động vào tác nhân nào, thông tin nào được trao đổi.

Quá trình trên được tiến hành với tất cả các chức năng yêu cầu của hệ thống.

Bước 3: Trích chọn đối tượng

Bước này sẽ tiến hành đề xuất các đối tượng có thể có mặt trong hệ thống:

Dựa vào các kịch bản được mô tả trong bước hai, chọn ra các tác nhân có xuất hiện để đề xuất thành các đối tượng.

Lựa chọn các đối tượng bằng cách loại bỏ các tác nhân bên ngoài hệ thống, các tác nhân trùng lặp.

Cuối cùng, ta thu được tập các đối tượng của hệ thống.

Bước 4: Mô hình hoá lớp đối tượng

Bước này tiến hành trừu tượng hoá đối tượng thành các lớp:

Thu thập tất cả các thuộc tính của mỗi đối tượng vừa thu thập được, dựa vào yêu cầu về thông tin trong yêu cầu hệ thống (từ bước 1).

Thu thập các hành động mà mỗi đối tượng cần thực hiện, dựa vào các kịch bản mà đối tượng tương ứng có tham gia (trong bước 2).

Nhóm các đối tượng tương tự nhau, hoặc có nhiều thuộc tính gần giống nhau.

Loại bỏ một số thuộc tính cá biệt, riêng tư của một số đối tượng trong nhóm.

Mô hình mỗi nhóm đối tượng còn lại thành lớp: Các thuộc tính chung của các đối tượng thành thuộc tính của lớp, các hành động của các đối tượng thành phương thức của lớp.

Kết quả thu được một tập các lớp đối tượng ban đầu của hệ thống.

Bước 5: Thiết kế tổng quát

Bước này sẽ tiến hành thiết kế vĩ mô, nghĩa là thiết kế mối quan hệ giữa các lớp trong hệ thống:

Xác định sơ đồ thừa kế, nếu có, giữa các lớp: Nếu hai lớp có một số thuộc tính chung, thì tách các thuộc tính chung làm thành một lớp cơ sở, và hai lớp ban đầu đều dẫn xuất từ lớp cơ sở đó. Thông thường, lớp các trừu tượng (chung nhất) sẽ làm lớp cơ sở, lớp càng cụ thể, càng chi tiết thì làm lớp dẫn xuất (lớp con, cháu).

Xác định tương tác, nếu có, giữa các lớp: Dựa vào các kịch bản được mô tả trong bước 2, hai tác nhân có tương tác với nhau thì hai lớp tương ứng ở bước này cũng có tương tác với nhau.

Kết quả thu được của bước này là một sơ đồ quan hệ bên ngoài giữa các lớp trong hệ thống.

Bước 6: Thiết kế chi tiết

Bước này sẽ thực hiện thiết kế ở mức vi mô, nghĩa là thiết kế kiến trúc bên trong của mỗi lớp đối tượng:

Tổ chức dữ liệu của lớp theo các thuộc tính. Qui định phạm vi truy nhập cho từng thuộc tính.

Thiết kế chi tiết cách xử của lớp đối tượng thông qua các phương thức của lớp: Xác định kiểu dữ liệu trả về, kiểu tham số của phương thức, mô tả thuật toán chi tiết cho từng phương thức, nếu cần.

Kết quả thu được của bước này là một tập các lớp với thiết kế chi tiết kiến trúc bên trong.

Sau các bước phân tích thiết kế hướng đối tượng từ một yêu cầu của bài toán ban đầu, ta thu được một mô hình hệ thống hướng đối tượng chi tiết:

Có cái nhìn tổng quan, vĩ mô về hệ thống bằng mô hình thiết kế tổng quan, chỉ rõ số lượng các lớp đối tượng, mối quan hệ kế thừa và quan hệ tương tác giữa các lớp đối tượng trong hệ thống.

Có cái nhìn chi tiết, vi mô về hệ thống bằng mô hình thiết kế chi tiết. Mô hình này chỉ rõ bên trong mỗi lớp đối tượng: các thuộc tính, các phương thức với kiểu trả về và kiểu tham số, thuật toán chi tiết cho mỗi phương thức.

Sau pha phân tích và thiết kế hướng đối tượng, ta thu được đặc tả hệ thống dưới dạng mô hình các lớp: quan hệ giữa các lớp và kiến trúc bên trong của mỗi lớp. Đây sẽ là đầu vào cho pha tiếp theo, pha lập trình hướng đối tượng, như chúng ta đã biết.

1.3 SO SÁNH HAI CÁCH TIẾP CẬN

Phương pháp tiếp cận hướng đối tượng có bản chất hoàn toàn khác với phương pháp tiếp cận truyền thống (phương pháp tiếp cận hướng cấu trúc) trên nhiều mặt:

Phương pháp mô hình bài toán khác nhau.

Đặc trưng khác nhau về đóng gói

Ưu / nhược điểm khác nhau.

Lĩnh vực ứng dụng khác nhau.

Khác nhau về phương pháp mô hình

Hai phương pháp này khác nhau hoàn toàn ở cách tiếp cận và mô hình bài toán, phương pháp hướng đối tượng tiến hành theo phương pháp từ dưới lên trên, từ thấp lên cao, từ cụ thể đến trừu tượng. Trong khi đó, phương pháp cấu trúc tiếp cận theo phương pháp từ trên xuống dưới, từ tổng quan đến chi tiết:

Phương pháp hướng đối tượng bắt đầu bằng những đối tượng cụ thể, tập hợp các thuộc tính của từng đối tượng. Sau đó, nhóm các đối tượng tương tự nhau thành nhóm, loại bỏ các thuộc tính quá cá biệt, chỉ giữ lại các thuộc tính chung nhất, nhóm thành lớp.

Cho nên, quá trình hình thành lớp là quá trình đi từ thấp lên cao, từ cụ thể ở mức thấp đến trừu tượng hoá ở mức cao.

Trong khi đó, phương pháp hướng cấu trúc lại đi theo chiều ngược lại. Phương pháp này bắt đầu từ một bài toán tổng quan, ở mức khái quát cao, chia nhỏ dần và làm mịn dần cho đến khi thu được một tập các bài toán con, nhỏ hơn, cụ thể hơn, chi tiết hơn.

Khác nhau về đặc trưng đóng gói

Hai phương pháp tiếp cận này cũng có những đặc trưng hoàn toàn khác nhau:

Phương pháp hướng đối tượng có đặc trưng là dữ liệu được đóng gói để hạn chế truy nhập tự do trực tiếp vào dữ liệu. Thứ hai là cho phép sử dụng lại mã nguồn để tiết kiệm tài nguyên và công sức lập trình.

Trong khi đó, đặc trưng của phương pháp cấu trúc là cấu trúc dữ liệu và giải thuật và mối quan hệ phụ thuộc chặt chẽ của giải thuật vào cấu trúc dữ liệu.

Khác nhau về ưu nhược điểm

Hai phương pháp này cũng có những ưu nhược điểm trái ngược nhau:

Phương pháp hướng đối tượng có ưu điểm là bảo vệ được dữ liệu tránh bị truy nhập trực tiếp tự do từ bên ngoài, tiết kiệm được tài nguyên và công sức lập trình do có thể dùng lại mã nguồn. Tuy nhiên, phương pháp này lại khá phức tạp, khó theo dõi được luồng dữ liệu và hơn nữa, giải thuật không phải là vấn đề trọng tâm của phương pháp này.

Trái lại, phương pháp hướng cấu trúc lại có ưu điểm là tư duy giải thuật rõ ràng, dễ theo dõi luồng dữ liệu, chương trình đơn giản và dễ hiểu. Tuy nhiên, không bảo vệ được an toàn dữ liệu trong chương trình. Hơn nữa, hạn chế lớn nhất là sự phụ thuộc chặt chẽ của giải thuật vào cấu trúc dữ liệu, khiến cho khi thay đổi cấu trúc dữ liệu, thường phải thay đổi giải thuật, và do đó, phải viết lại mã cho chương trình.

Khác nhau về lĩnh vực áp dụng

Do sự khác nhau về các đặc trưng và sự khác nhau về ưu nhược điểm, cho nên hai phương pháp này cũng có sự khác nhau đáng kể trong lĩnh vực áp dụng:

Phương pháp hướng đối tượng thường được áp dụng cho các bài toán lớn, phức tạp, có nhiều luồng dữ liệu khác nhau, không thể quản lý được bằng phương pháp cấu trúc. Khi đó, người ta dùng phương pháp hướng đối tượng để tận dụng khả năng bảo vệ dữ liệu tránh bị truy nhập tự do. Hơn nữa, tận dụng khả năng dùng lại mã nguồn của phương pháp này để tiết kiệm tài nguyên và công sức.

Trong khi đó, phương pháp cấu trúc thường phù hợp với các bài toán nhỏ, có luồng dữ liệu rõ ràng, cần phải tư duy giải thuật rõ ràng và người lập trình vẫn có khả năng tự quản lý được mọi truy nhập đến các dữ liệu của chương trình.

1.4 XU HƯỚNG PHÁT TRIỂN CỦA LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Lập trình hướng thành phần

Xuất phát từ lập trình hướng đối tượng, tư duy lập trình hướng thành phần (component-oriented programming) theo ý tưởng:

Giải quyết bài toán bằng cách xây dựng một tập các thành phần (component) có tính độc lập tương đối với nhau. Mỗi thành phần đảm nhiệm một phần công việc nhất định.

Sau đó, người ta ghép các thành phần với nhau để thu được một phần mềm thoả mãn một tập các yêu cầu xác định.

Với lập trình hướng thành phần, người ta có thể tiến hành lập trình theo phương pháp sau:

Xây dựng một thư viện các thành phần, mỗi thành phần thực hiện một công việc xác định.

Khi cần phát triển một phần mềm cụ thể, người ta chỉ cần chọn những thành phần có sẵn trong thư viện để ghép lại với nhau. Người lập trình chỉ phải phát triển thêm các thành phần mình cần mà chưa có trong thư viện.

Phương pháp này có những ưu điểm rất lớn:

Lập trình viên có thể chia sẻ với nhau những thành phần mình đã xây dựng cho nhiều người khác dùng chung.

Khi cần, lập trình viên có thể lắp ghép các thành phần có sẵn khác nhau để tạo thành các chương trình có chức năng khác nhau. Tất cả chỉ cần dựa trên công nghệ lắp ghép thành phần, tiết kiệm được rất nhiều công sức lập trình.

Trong xu hướng lập trình hướng thành phần, một số phương pháp lập trình khác đã nảy sinh và đang phát triển mạnh mẽ:

Lập trình hướng agent (agent-oriented programming)

Lập trình hướng aspect (aspect-oriented programming)

Lập trình hướng agent

Lập trình hướng agent có thể xem là một mức trừu tượng cao hơn của lập trình hướng thành phần. Trong đó, các agent là các thành phần có khả năng hoạt động độc lập, tự chủ để hoàn thành công việc của mình. Hơn nữa, các agent có khả năng chủ động liên lạc với các agent khác để có thể phối hợp, cộng tác hay cạnh tranh nhau để hoàn thành nhiệm vụ.

Lập trình hướng agent có hai đặc trưng cơ bản:

Thứ nhất là khả năng tự chủ của mỗi agent để hoàn thành nhiệm vụ riêng của nó.

Thứ hai là tính tổ chức xã hội giữa các agent, cho phép các agent phối hợp, cộng tác, cạnh tranh nhau để hoàn thành nhiệm vụ chung của toàn hệ thống.

Lập trình hướng aspect

Lập trình hướng aspect cũng là một xu hướng của lập trình hướng thành phần. Theo đó, mỗi thành phần có nhiệm vụ hoàn thành theo một luồng công việc hoặc một khía cạnh của vấn đề. Sau đó, tổng hợp các thành phần của các luồng khác nhau, ta thu được giải pháp cho bài toán của mình.

Lập trình hướng aspect có đặc trưng cơ bản:

Tính đóng gói theo luồng công việc, hoặc đóng gói theo khía cạnh của vấn đề.

Tính đơn điệu theo luồng, trong một luồng công việc, các nhiệm vụ được thực hiện liên tiếp nhau, tuần tự như trong lập trình tuyến tính.

TỔNG KẾT CHƯƠNG 1

Nội dung chương 1 đã trình bày các vấn đề tổng quan liên quan đến phương pháp tiếp cận hướng đối tượng trong lập trình:

Các phương pháp tiếp cận truyền thống: lập trình tuyến tính và lập trình cấu trúc.

Phương pháp tiếp cận hướng đối tượng với hai đặc trưng cơ bản: Đóng gói dữ liệu và sử dụng lại mã nguồn.

Lập trình hướng đối tượng, phương pháp phân tích và thiết kế hệ thống hướng đối tượng.

So sánh sự khác biệt của phương pháp hướng đối tượng với các phương pháp truyền thống trên các khía cạnh: Cách tiếp cận bài toán, đặc trưng, ưu nhược điểm và lĩnh vực áp dụng của mỗi phương pháp.

Hiện nay, lập trình hướng thành phần, lập trình hướng agent và lập trình hướng aspect tiến hoá từ lập trình hướng đối tượng đang là xu hướng phát triển mạnh mẽ.

CHƯƠNG 2

NHỮNG KHÁI NIỆM CƠ BẢN CỦA

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Nội dung chương này tập trung trình bày các khái niệm cơ bản của lập trình hướng đối tượng:

Đối tượng

Lớp đối tượng

Việc trừu tượng hoá đối tượng theo chức năng

Trừu tượng hoá đối tượng theo dữ liệu

Kế thừa

Đóng gói

Đa hình

Phương pháp cài đặt lớp đối tượng

Giới thiệu một số ngôn ngữ lập trình hướng đối tượng thông dụng hiện nay.

2.1 CÁC KHÁI NIỆM CƠ BẢN

2.1.1 Đối tượng

Trong lập trình hướng đối tượng, tất cả các thực thể trong hệ thống đều được coi là các đối tượng cụ thể. Đối tượng là một thực thể hoạt động khi chương trình đang chạy.

Ví dụ:

Trong bài toán quản lý buôn bán xe hơi của một cửa hàng kinh doanh, mỗi chiếc xe đang có mặt trong cửa hàng được coi là một đối tượng. Chẳng hạn, một chiếc xe nhãn hiệu “Ford”, màu trắng, giá 5000\$ là một đối tượng.

Trong bài toán quản lý nhân viên của một văn phòng, mỗi nhân viên trong văn phòng được coi là một đối tượng. Chẳng hạn, nhân viên tên là “Vinh”, 25 tuổi làm ở phòng hành chính là một đối tượng.

Một đối tượng là một thực thể đang tồn tại trong hệ thống và được xác định bằng ba yếu tố:

Định danh đối tượng: xác định duy nhất cho mỗi đối tượng trong hệ thống, nhằm phân biệt các đối tượng với nhau.

Trạng thái của đối tượng: là sự tổ hợp của các giá trị của các thuộc tính mà đối tượng đang có.

Hoạt động của đối tượng: là các hành động mà đối tượng có khả năng thực hiện

được.

Trạng thái hiện tại của đối tượng qui định tính chất đặc trưng của đối tượng. Ví dụ, nhân viên trong ví dụ trên có trạng thái là:

Tên là Vinh

Tuổi là 25

Vị trí làm việc là phòng hành chính.

Trong khi đó, trạng thái của chiếc xe trong cửa hàng là:

Nhãn hiệu xe là Ford

Màu sơn xe là trắng

Giá bán xe là 5000\$

Mỗi đối tượng sẽ thực hiện một số hành động. Ví dụ, đối tượng xe hơi có khả năng thực hiện những hành động sau:

Khởi động.

Dừng lại.

Chạy.

Để biểu diễn đối tượng trong lập trình hướng đối tượng, người ta trừu tượng hoá đối tượng để tạo nên khái niệm lớp đối tượng.

2.1.2 Lớp đối tượng

Trong lập trình hướng đối tượng, đối tượng là một thực thể cụ thể, tồn tại trong hệ thống. Trong khi đó, lớp là một khái niệm trừu tượng, dùng để chỉ một tập hợp các đối tượng có mặt trong hệ thống.

Ví dụ:

Trong bài toán quản lý buôn bán xe hơi của một cửa hàng kinh doanh, mỗi chiếc xe đang có mặt trong cửa hàng được coi là một đối tượng. Nhưng khái niệm “Xe hơi” là một lớp đối tượng dùng để chỉ tất cả các loại xe hơi của cửa hàng.

Trong bài toán quản lý nhân viên của một văn phòng, mỗi nhân viên trong văn phòng được coi là một đối tượng. Nhưng khái niệm “Nhân viên” là một lớp đối tượng dùng để chỉ chung chung các nhân viên của văn phòng.

Lưu ý:

Lớp là một khái niệm, mang tính trừu tượng, dùng để biểu diễn một tập các đối tượng.

Đối tượng là một thể hiện cụ thể của lớp, là một thực thể tồn tại trong hệ thống.

Lớp được dùng để biểu diễn đối tượng, cho nên lớp cũng có thuộc tính và phương thức:

Thuộc tính của lớp tương ứng với thuộc tính của các đối tượng.

Phương thức của lớp tương ứng với các hành động của đối tượng.

Ví dụ, lớp xe ô tô được mô tả bằng các thuộc tính và phương thức:

Lớp Xe ô tô

Thuộc tính:

Nhãn hiệu xe

Màu xe

Giá xe

Công suất xe (mã lực)

Phương thức:

Khởi động xe

Chạy xe

Dừng xe

Tắt máy

Lưu ý:

Một lớp có thể có một trong các khả năng sau:

Hoặc chỉ có thuộc tính, không có phương thức.

Hoặc chỉ có phương thức, không có thuộc tính.

Hoặc có cả thuộc tính và phương thức, trường hợp này là phổ biến nhất.

Đặc biệt, lớp không có thuộc tính và phương thức nào là các lớp trừu tượng. Các lớp này không có đối tượng tương ứng.

Lớp và Đối tượng

Lớp và đối tượng, mặc dù có mối liên hệ tương ứng lẫn nhau, nhưng bản chất lại khác nhau:

Lớp là sự trừu tượng hoá của các đối tượng. Trong khi đó, đối tượng là một thể hiện của lớp.

Đối tượng là một thực thể cụ thể, có thực, tồn tại trong hệ thống. Trong khi đó, lớp là một khái niệm trừu tượng, chỉ tồn tại ở dạng khái niệm để mô tả các đặc tính chung của một số đối tượng.

Tất cả các đối tượng thuộc về cùng một lớp có cùng các thuộc tính và các phương thức.

Một lớp là một nguyên mẫu của một đối tượng. Nó xác định các hành động khả thi và các thuộc tính cần thiết cho một nhóm các đối tượng cụ thể.

Nói chung, lớp là khái niệm tồn tại khi phát triển hệ thống, mang tính khái niệm, trừu tượng. Trong khi đó, đối tượng là một thực thể cụ thể tồn tại khi hệ thống đang hoạt động.

2.1.3 Trừu tượng hoá đối tượng theo chức năng

Trừu tượng hoá đối tượng theo chức năng chính là quá trình mô hình hoá phương thức của lớp dựa trên các hành động của các đối tượng. Quá trình này được tiến hành như sau:

Tập hợp tất cả các hành động có thể có của các đối tượng.

Nhóm các đối tượng có các hoạt động tương tự nhau, loại bỏ bớt các hoạt động cá biệt, tạo thành một nhóm chung.

Mỗi nhóm đối tượng đề xuất một lớp tương ứng.

Các hành động chung của nhóm đối tượng sẽ cấu thành các phương thức của lớp tương ứng.

Ví dụ, trong bài toán quản lý cửa hàng bán ô tô. Mỗi ô tô có mặt trong cửa hàng là một đối tượng. Mặc dù mỗi chiếc xe có một số đặc điểm khác nhau về nhãn hiệu, giá xe, màu sắc... nhưng có chung các hành động của một chiếc xe ô tô là:

Có thể khởi động máy.

Có thể chạy.

Có thể dừng lại.

Có thể tắt máy.

Ngoài ra, một số ít xe có thể thực hiện một số hành động cá biệt như:

Có thể giấu đèn pha

Có thể tự bật đèn pha

Có thể tự động phát tín hiệu báo động.

Tuy nhiên, không phải xe nào cũng thực hiện được các hành động này. Cho nên ta loại bỏ các hành động cá biệt của một số xe, chỉ giữ lại các hành động chung nhất, để mô hình thành các phương thức của đối tượng xe ô tô tương ứng với các hành động chung nhất của các xe ô tô.

Lớp Xe ô tô

Phương thức:

Khởi động xe

Chạy xe

Dừng xe

Tắt máy

2.1.4 Trừu tượng hoá đối tượng theo dữ liệu

Trừu tượng hoá đối tượng theo dữ liệu chính là quá trình mô hình hoá các thuộc tính của lớp dựa trên các thuộc tính của các đối tượng tương ứng. Quá trình này được tiến hành như sau:

Tập hợp tất cả các thuộc tính có thể có của các đối tượng.

Nhóm các đối tượng có các thuộc tính tương tự nhau, loại bỏ bớt các thuộc tính cá biệt, tạo thành một nhóm chung.

Mỗi nhóm đối tượng đề xuất một lớp tương ứng.

Các thuộc tính chung của nhóm đối tượng sẽ cấu thành các thuộc tính tương ứng của lớp được đề xuất.

Ví dụ, trong bài toán quản lí cửa hàng bán ô tô. Mỗi ô tô có mặt trong cửa hàng là một đối tượng. Mặc dù mỗi chiếc xe có một số đặc điểm khác nhau về nhãn hiệu, giá xe, màu sắc... nhưng có chung các thuộc tính của một chiếc xe ô tô là:

Các xe đều có nhãn hiệu.

Các xe đều có màu sắc

Các xe đều có giá bán

Các xe đều có công suất động cơ

Ngoài ra, một số ít xe có thể có thêm các thuộc tính:

Có xe có thể có dàn nghe nhạc

Có xe có thể có màn hình xem tivi

Có xe có lắp kính chống nắng, chống đạn...

Tuy nhiên, đây là các thuộc tính cá biệt của một số đối tượng xe, nên không được đề xuất thành thuộc tính của lớp ô tô. Do đó, ta mô hình lớp ô tô với các thuộc tính chung nhất của các ô tô.

Lớp Xe ô tô

Thuộc tính:

Nhãn hiệu xe

Màu xe

Giá xe

Công suất xe (mã lực)

Ưu điểm của việc trừu tượng hóa

Những ưu điểm của việc trừu tượng hóa là:

Tập trung vào vấn đề cần quan tâm

Xác định những đặc tính thiết yếu và những hành động cần thiết

Giảm thiểu những chi tiết không cần thiết

Việc trừu tượng hóa dữ liệu là cần thiết, bởi vì không thể mô tả tất cả các hành động và các thuộc tính của một thực thể. Vấn đề mấu chốt là tập trung đến những hành vi cốt yếu và áp dụng chúng trong ứng dụng.

2.1.5 Khái niệm kế thừa

Xét trường hợp bài toán quản lý nhân sự và sinh viên của một trường đại học. Khi đó, ta có hai lớp đối tượng chính là lớp Nhân viên và lớp Sinh viên:

<i>Lớp Nhân viên</i>	<i>Lớp Sinh viên</i>
Thuộc tính:	Thuộc tính:
Tên	Tên
Ngày sinh	Ngày sinh
Giới tính	Giới tính
Lương	Lớp
Phương thức:	Phương thức:
Nhập/xem tên	Nhập/xem tên
Nhập/xem ngày sinh	Nhập/xem ngày sinh
Nhập/xem giới tính	Nhập/xem giới tính
Nhập/xem lương	Nhập/xem lớp

Ta nhận thấy rằng hai lớp này có một số thuộc tính và phương thức chung: tên, ngày sinh, giới tính. Tuy nhiên, không thể loại bỏ các thuộc tính cá biệt để gộp chúng thành một lớp duy nhất, vì các thuộc tính lương nhân viên và lớp của sinh viên là cần thiết cho việc quản lý. Vấn đề nảy sinh như sau:

Ta phải viết mã trùng nhau đến hai lần cho các phương thức: nhập/xem tên, nhập/xem ngày sinh, nhập/xem giới tính. Rõ ràng điều này rất tốn công sức.

Nếu khi có sự thay đổi về kiểu dữ liệu, chẳng hạn kiểu ngày sinh được quản lý trong hệ thống, ta phải sửa lại chương trình hai lần.

Để tránh rắc rối do các vấn đề nảy sinh như vậy, lập trình hướng đối tượng sử dụng kỹ thuật kế thừa nhằm nhóm các phần giống nhau của các lớp thành một lớp mới, sau đó cho các lớp ban đầu kế thừa lại lớp được tạo ra. Như vậy, mỗi lớp thừa kế (lớp dẫn xuất, lớp

con) đều có các thuộc tính và phương thức của lớp bị thừa kế (lớp cơ sở, lớp cha).

Quay lại với bài toán quản lí trường đại học, các thuộc tính và phương thức chung giữa lớp Nhân viên và lớp Sinh viên là:

Tên,

Ngày sinh,

Giới tính,

Nhập/xem tên,

Nhập/xem ngày sinh

Nhập/xem giới tính.

Ta tách phần chung này thành một lớp ở mức trừu tượng cao hơn, lớp Người. Lớp Người sẽ làm lớp cha của lớp Nhân viên và lớp Sinh viên. Khi đó, các lớp trở thành:

Lớp Người Thuộc tính: Tên Ngày sinh Giới tính Phương thức: Nhập/xem tên Nhập/xem ngày sinh Nhập/xem giới tính	
Lớp Nhân viên kế thừa từ lớp Người Thuộc tính: Lương Phương thức: Nhập/xem lương	Lớp Sinh viên kế thừa từ lớp Người Thuộc tính: Lớp Phương thức: Nhập/xem lớp

Như vậy, sự kế thừa trong lập trình hướng đối tượng:

Cho phép lớp dẫn xuất có thể sử dụng các thuộc tính và phương thức của lớp cơ sở tương tự như sử dụng các thuộc tính và phương thức của mình.

Cho phép việc chỉ cần cài đặt phương thức ở một lớp cơ sở, mà có thể sử dụng được ở tất cả các lớp dẫn xuất.

Cho phép tránh sự cài đặt trùng lặp mã nguồn của chương trình.

Cho phép chỉ phải thay đổi một lần khi cần phải thay đổi dữ liệu của các lớp.

2.1.6 Khái niệm đóng gói

Xét ví dụ bài toán quản lý nhân viên văn phòng với lớp Nhân viên như sau:

Lớp Nhân viên
Thuộc tính:
Tên
Ngày sinh
Giới tính
Phòng ban
Hệ số lương
Phương thức:
Tính lương nhân viên

Khi đó, cách tính lương cho nhân viên là khác nhau đối với mỗi người:

$$\langle \text{Tiền lương} \rangle = \langle \text{Hệ số lương} \rangle * \langle \text{Lương cơ bản} \rangle * \langle \text{Tỉ lệ phần trăm} \rangle$$

Trong đó, tỉ lệ phần trăm là khác nhau cho mỗi phòng ban, ví dụ:

Phòng kế hoạch là 105%

Phòng hành chính là 100%

Phòng nhân sự là 110%

Khi đó, tùy vào thuộc tính phòng ban khác nhau mà ta phải dùng công thức tỉ lệ khác nhau để tính lương cho mỗi nhân viên.

Tuy nhiên, cách tính cụ thể này là công việc bên trong của phương thức tính tiền lương của lớp Nhân viên. Với mỗi ứng dụng, khi tạo một đối tượng cụ thể của lớp nhân viên, ta chỉ cần truyền các tham số thuộc tính cho đối tượng, sau đó gọi phương thức tính tiền lương cho đối tượng nhân viên đó, ta sẽ biết được tiền lương của nhân viên. Cách gọi phương thức tính tiền lương là hoàn toàn giống nhau cho tất cả các đối tượng nhân viên của văn phòng.

Sự giống nhau về cách sử dụng phương thức cho các đối tượng của cùng một lớp, mặc dù bên trong phương thức có các cách tính toán khác nhau với các đối tượng khác nhau, được gọi là tính đóng gói dữ liệu của lập trình hướng đối tượng. Như vậy, tính đóng gói dữ liệu của lập trình hướng đối tượng:

Cho phép che dấu sự cài đặt chi tiết bên trong của phương thức. Khi sử dụng chỉ cần gọi các phương thức theo một cách thống nhất, mặc dù các phương thức có thể được cài đặt khác nhau cho các trường hợp khác nhau.

Cho phép che dấu dữ liệu bên trong của đối tượng. Khi sử dụng, ta không biết được thực sự bên trong đối tượng có những gì, ta chỉ thấy được những gì đối tượng cho phép truy nhập vào.

Cho phép hạn chế tối đa việc sửa lại mã chương trình. Khi phải thay đổi công thức tính toán của một phương thức, ta chỉ cần thay đổi mã bên trong của phương thức, mà không phải thay đổi các chương trình gọi đến phương thức bị thay đổi.

2.1.7 Khái niệm đa hình

Trở lại với ví dụ về quản lý trường đại học, với hai lớp Nhân viên và lớp Sinh viên, đều kế thừa từ lớp Người. Khi đó, ta thêm vào mỗi lớp một phương thức show():

Phương thức show của lớp Người sẽ giới thiệu tên và tuổi của người đó.

Phương thức show của lớp Nhân viên sẽ giới thiệu nhân viên đó có tiền lương là bao nhiêu

Phương thức show của lớp Sinh viên sẽ giới thiệu là sinh viên đó đang học ở lớp nào.

Lớp Người Thuộc tính: Tên Ngày sinh Giới tính Phương thức: Nhập/xem tên Nhập/xem ngày sinh Nhập/xem giới tính show	
Lớp Nhân viên kế thừa từ lớp Người Thuộc tính: Lương Phương thức: Nhập/xem lương show	Lớp Sinh viên kế thừa từ lớp Người Thuộc tính: Lớp Phương thức: Nhập/xem lớp show

Khi đó, nếu trong hệ thống có các đối tượng cụ thể tương ứng với ba lớp, thì:

Khi ta gọi hàm show từ đối tượng của lớp Người, sẽ nhận được tên và tuổi của người đó.

Khi ta gọi phương thức show từ đối tượng của lớp Nhân viên, sẽ nhận được số tiền lương của nhân viên đó.

Khi ta gọi phương thức show từ đối tượng của lớp Sinh viên, ta sẽ biết được lớp học của sinh viên đó.

Việc chỉ cần gọi cùng một phương thức, nhưng từ các đối tượng khác nhau, sẽ cho kết quả khác nhau được gọi là tính đa hình trong lập trình hướng đối tượng. Như vậy, tính đa hình trong lập trình hướng đối tượng:

Cho phép các lớp được định nghĩa các phương thức trùng nhau: cùng tên, cùng số lượng và kiểu tham số, cùng kiểu trả về. Việc định nghĩa phương thức trùng nhau của các lớp kế thừa nhau còn được gọi là sự nạp chồng phương thức.

Khi gọi các phương thức trùng tên, dựa vào đối tượng đang gọi mà chương trình sẽ thực hiện phương thức của lớp tương ứng, và do đó, sẽ cho các kết quả khác nhau.

2.2 SO SÁNH LỚP VÀ CẤU TRÚC

Trong phần này, chúng ta sẽ tiến hành so sánh Class (Lớp) và Structure (Cấu trúc) trên nhiều khía cạnh khác nhau:

Mức khái niệm

Mục đích và chức năng

Về ưu và nhược điểm

So sánh ở mức khái niệm

mức khái niệm, Lớp và cấu trúc hoàn toàn khác nhau:

Lớp là khái niệm chỉ có trong lập trình hướng đối tượng; nó được dùng để biểu diễn một tập các đối tượng tương tự nhau.

Trong khi đó, Cấu trúc là khái niệm chỉ tồn tại trong lập trình cấu trúc, không phải là một khái niệm của lập trình hướng đối tượng. Vì trong lập trình hướng đối tượng, tất cả các thực thể đều được coi là một đối tượng, nghĩa là nó phải là một thể hiện cụ thể của một lớp nào đó. Do đó, trong lập trình hướng đối tượng, không có khái niệm Cấu trúc.

So sánh về mục đích và chức năng

Về mục đích, Lớp và Cấu trúc đều có chung một mục đích ban đầu, đó là nhóm một tập hợp các dữ liệu lại với nhau để xử lý đồng bộ và thống nhất: Cấu trúc nhóm các dữ liệu hay phải đi kèm với nhau lại thành một nhóm cho dễ xử lý. Tương tự, Lớp là tập hợp một số thuộc tính chung của đối tượng để xử lý.

Tuy nhiên, Lớp và Cấu trúc cũng có một số khác biệt trên khía cạnh này:

Lớp ngoài mục đích nhóm các thuộc tính dữ liệu của đối tượng, còn nhóm các hoạt động của đối tượng thành các phương thức của Lớp.